ESP32-S3 (4848S040) Dual-Core Architecture

Technical Specifications: LVGL v8.4 & Custom C-Based Forth Integration

1. Core Allocation & Performance Topology

To optimize performance on the 4848S040 480x480 RGB display, system tasks are separated by execution requirements:

- **Core 0 (System Backend):** Runs the ESP-IDF Wi-Fi/Bluetooth stack at high priority (18–23) and the custom C Forth VM at low priority (5). FreeRTOS preemption ensures long-running Forth words yield instantly to network traffic without disrupting connectivity.
- **Core 1 (Graphics Frontend):** Dedicated exclusively to the LVGL v8.4 engine loop at priority 10. This avoids UI stutter by insulating layout computations and DMA transfers from background calculations.

2. Memory Management Strategy

- **Internal SRAM (MALLOC_CAP_INTERNAL):** Allocates Forth data/return stacks and FreeRTOS queues. Fast, zero bus-contention layout guarantees deterministic operation.
- **External PSRAM (MALLOC_CAP_SPIRAM):** Stores massive framebuffers and the LVGL canvas layer. Forth limits PSRAM access to large dynamic memory structures to avoid memory bus saturation.

3. Dual-Queue Inter-Core Messaging Layout

All data cross-core boundary operations are managed through two distinct FreeRTOS queues, guaranteeing strict thread safety without shared memory hazards.

A. Front-End Command Queue (Core 0 → Core 1)

Passes abstract vector/Logo drawing commands or widget adjustments. Forth on Core 0 evaluates math and coordinates, transforming operations into low-overhead memory structures.

```
// 1. Data Structures for Logo Vector Commands & Widget Control
typedef enum {
    LOGO_CLEAR_SCREEN,
    LOGO_PEN_COLOR,
    LOGO_PEN_WIDTH,
    LOGO_DRAW_LINE,
    LOGO_DRAW_CIRCLE,
    CMD_UI_CREATE_BTN,
    CMD_UI_SET_VALUE
} logo_op_t;

typedef struct {
    uint8_t  op_code;    // Operation identifier (logo_op_t)
    uint16_t target_id;  // Unique ID for widget references
    int16_t  x1;         // X1 coordinate or geometry argument
    int16_t  y1;         // Y1 coordinate or geometry argument
    int16_t  x2;         // X2 coordinate / object width
    int16_t  y2;         // Y2 coordinate / object height
    uint32_t extra;      // Raw 16-bit or 24-bit color maps
} logo_cmd_t;
```

B. Back-End Feedback Queue (Core 1 → Core 0)

Pushes touchscreen interactions from the UI layer back to the interpreter environment.

```
// 2. Data Structures for UI Input Processing
typedef enum {
    EVENT_BTN_CLICKED,
    EVENT_SLIDER_CHANGED
} event_id_t;

typedef struct {
    uint16_t widget_id;   // Associated UI element tracker mapping identifier
    uint8_t  event_type;  // Interaction context category (event_id_t)
    int32_t  value;       // Absolute value offset payload data
} ui_feedback_t;
```

4. Bridge Execution Mechanics

A. Dynamic Mapping Mechanism

To coordinate button clicks or slider tracking without executing Forth code directly within Core 1 context, a cross-reference matrix tracks allocations:

```
#define MAX_UI_ELEMENTS 32

typedef struct {
    uint16_t id;         // Dynamic execution parameter ID
    uint32_t forth_xt;   // Target compiled Forth Execution Token (XT)
    lv_obj_t *lv_obj;    // Target instanced visual asset reference context
} ui_map_t;

ui_map_t ui_registry[MAX_UI_ELEMENTS];
int ui_registry_count = 0;
```

5. Runtime Application Code Implementation

A. Forth Interface C Primitive (Core 0 Placement)

```
// Exposed to Forth engine dictionary as: CREATE-BUTTON
void forth_primitive_create_button() {
    uint32_t xt    = (uint32_t)forth_pop();
    uint16_t id    = (uint16_t)forth_pop();
    int16_t  h     = (int16_t)forth_pop();
    int16_t  w     = (int16_t)forth_pop();
    int16_t  y     = (int16_t)forth_pop();
    int16_t  x     = (int16_t)forth_pop();

    if (ui_registry_count < MAX_UI_ELEMENTS) {
        ui_registry[ui_registry_count].id = id;
        ui_registry[ui_registry_count].forth_xt = xt;
        ui_registry_count++;
    }

    logo_cmd_t cmd = {
        .op_code = CMD_UI_CREATE_BTN,
        .target_id = id,
        .x1 = x, .y1 = y, .x2 = w, .y2 = h
    };
    xQueueSend(logo_queue, &cmd;, portMAX_DELAY);
}
```

B. Consumer Rendering Loop (Core 1 Placement - LVGL v8.4)

```
void lvgl_core1_drawing_task(void *pvParameters) {
    logo_cmd_t cmd;
    lv_draw_line_dsc_t line_dsc;
    lv_draw_line_dsc_init(&line;_dsc);

    while(1) {
        while (xQueueReceive(logo_queue, &cmd;, 0) == pdTRUE) {
            switch(cmd.op_code) {
                case LOGO_DRAW_LINE:
                    line_dsc.p1.x = cmd.x1; line_dsc.p1.y = cmd.y1;
                    line_dsc.p2.x = cmd.x2; line_dsc.p2.y = cmd.y2;
                    lv_canvas_draw_line(canvas_object, &line;_dsc);
                    break;
                case LOGO_CLEAR_SCREEN:
                    lv_canvas_fill_bg(canvas_object, lv_color_black(), LV_OPA_COVER);
                    break;
                case CMD_UI_CREATE_BTN:
                    lv_obj_t * btn = lv_btn_create(lv_scr_act());
                    lv_obj_set_coord(btn, cmd.x1, cmd.y1, cmd.x2, cmd.y2);
                    lv_obj_set_user_data(btn, (void*)(uintptr_t)cmd.target_id);
                    lv_obj_add_event_cb(btn, universal_btn_click_cb, LV_EVENT_CLICKED, NU
LL);
                    break;
            }
        }
        lv_timer_handler();
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}
```